

LogisticRegression

January 2, 2026

1 Logistic Regression

2 Project: Fraud Detection - Logistic Regression Baseline

Ngày thực hiện: -/12/2025

2.1 1. Mục tiêu & Bối cảnh

Trong bài toán phát hiện gian lận tài chính, chúng ta đối mặt với dữ liệu mất cân bằng nghiêm trọng (**Fraud rate** ~ **0.025%**). Mục tiêu của Notebook này là xây dựng một **Baseline Model** sử dụng Logistic Regression để: 1. Thiết lập mức hiệu suất cơ sở (Benchmark). 2. Đánh giá mối quan hệ tuyến tính (Linearity) giữa các biến và biến mục tiêu. 3. Xác định các đặc trưng (features) có trọng số cao nhất.

2.2 2. Chiến lược tiếp cận

- **Feature Engineering:** Log-transform biến tiền tệ (amount) để xử lý phân phối lệch (skewness).
- **Preprocessing:** Bắt buộc sử dụng `StandardScaler` để đưa các biến về cùng phạm vi, giúp thuật toán Gradient Descent hội tụ tốt hơn.
- **Imbalanced Handling:** Thử nghiệm SMOTE kết hợp ENN (Edited Nearest Neighbours) để làm sạch biên giới quyết định.

```
[3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
    cross_val_score
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    roc_auc_score, average_precision_score,
    precision_score, recall_score, f1_score,
```

```

        classification_report, confusion_matrix
    )

import optuna

```

2.3 Load dữ liệu

```

[4]: df = pd.read_csv("C://Users//hi//Downloads//fraud_detection_project//data//
    ↳processed//fraud_feature_engineering.csv")
df

```

```

[4]:
   Unnamed: 0  step  type  amount  nameOrig  oldbalanceOrig  \
0           0    401  CASH_IN  290670.88  C1172770329      30613326.59
1           1    346  CASH_OUT   74969.02   C361253330           0.00
2           2     34  CASH_OUT  152359.15  C1296741848           0.00
3           3    302  TRANSFER  553238.82   C765587136           0.00
4           4    130  CASH_IN  432624.19   C852698921      18910.00
...         ...   ...   ...   ...   ...   ...
325929      325929   282  CASH_OUT   34345.29   C943859349      20380.00
325930      325930   379  PAYMENT    8234.77  C1729134111       272.00
325931      325931   235  CASH_OUT   13499.45   C297276080           0.00
325932      325932   369  CASH_OUT  236678.98  C2094429825     10018.00
325933      325933   395  CASH_OUT  108989.28  C1625687711    337544.00

   newbalanceOrig  nameDest  oldbalanceDest  newbalanceDest  ...  \
0      30903997.48  C1360884032      6252769.10      5962098.22  ...
1           0.00  C1628643300      1510659.40      1585628.42  ...
2           0.00  C1234551068       519498.85       671857.99  ...
3           0.00   C845383981     1278572.18     1831811.01  ...
4      451534.19   C290472837       306529.18           0.00  ...
...         ...   ...   ...   ...   ...
325929           0.00  C1322395030           0.00      34345.29  ...
325930           0.00  M1758068394           0.00           0.00  ...
325931           0.00  C1955063491       84106.06      97605.52  ...
325932           0.00  C1407924212           0.00     236678.98  ...
325933     228554.72   C763824199     389833.54     478305.71  ...

   isCustomerDest  hour  day  abs_balance_diff  abs_errorBalanceDest  \
0                1    17   16      581341.77      5.813418e+05
1                1    10   14       74969.02      1.455192e-11
2                1    10    1      152359.15      1.000000e-02
3                1    14   12      553238.82      1.000000e-02
4                1    10    5      865248.38      7.391534e+05
...         ...   ...   ...   ...   ...
325929           1    18   11       13965.29      0.000000e+00
325930           0    19   15        7962.77      8.234770e+03
325931           1    19    9       13499.45      1.000000e-02

```

```

325932          1      9   15          226660.98          0.000000e+00
325933          1     11   16              0.00          2.051711e+04

      is_empty_orig  is_empty_dest  amount_ratio_org  log_amount  red_flag
0                0                0          0.009495    12.579950         1
1                1                0         74969.020000    11.224844         1
2                1                0        152359.150000    11.934002         1
3                1                0        553238.820000    13.223547         1
4                0                0         22.876854    12.977627         1
...
325929          0                1          1.685162    10.444249         1
325930          0                1          30.163993     9.016242         1
325931          1                0        13499.450000     9.510478         1
325932          0                1         23.623014    12.374464         1
325933          0                0          0.322888    11.599014         1

```

[325934 rows x 25 columns]

```
[19]: df.columns
```

```
[19]: Index(['Unnamed: 0', 'step', 'type', 'amount', 'nameOrig', 'oldbalanceOrig',
        'newbalanceOrig', 'nameDest', 'oldbalanceDest', 'newbalanceDest',
        'isFraud', 'isFlaggedFraud', 'balance_diff', 'errorBalanceDest',
        'isMerchant', 'isCustomerDest', 'hour', 'day', 'abs_balance_diff',
        'abs_errorBalanceDest', 'is_empty_orig', 'is_empty_dest',
        'amount_ratio_org', 'log_amount', 'red_flag'],
        dtype='object')
```

2.4 Danh sách Biến Cần GIỮ (Keep Features)

Các biến dưới đây được lựa chọn dựa trên **lý do nghiệp vụ rõ ràng**, thể hiện các giả thuyết về hành vi gian lận và phù hợp để sử dụng cho cả mô hình Tuyến tính (Linear) và Phi tuyến (Non-linear/Tree-based).

Tên Biến (Variable)	Lý do giữ & Hướng xử lý
type	Biến phân loại loại hình giao dịch. Cần mã hóa One-hot.
amount	Số tiền giao dịch. Giữ nguyên cho mô hình Cây (Tree), nên Log hóa cho mô hình Tuyến tính.
balance_diff	Thể hiện sự vi phạm nguyên tắc kế toán (số dư trước - sau - amount) tại tài khoản gốc.
errorBalanceDest	Tương tự balance_diff nhưng tính cho tài khoản đích (Destination).
abs_balance_diff	Độ lớn tuyệt đối của sai lệch kế toán. Đặc biệt hữu ích cho Logistic Regression.
abs_errorBalanceDest	Độ lớn tuyệt đối của sai lệch tại tài khoản đích.

Tên Biến (Variable)	Lý do giữ & Hướng xử lý
<code>is_empty_orig</code>	Cờ nhị phân (Binary Flag): Tài khoản gốc có số dư = 0 hay không.
<code>is_empty_dest</code>	Cờ nhị phân (Binary Flag): Tài khoản đích có số dư = 0 hay không.
<code>amount_ratio_org</code>	Tỷ lệ giao dịch so với số dư gốc. Đã chuẩn hóa theo ngưỡng cảnh.
<code>log_amount</code>	Biến đổi Log của <code>amount</code> để ổn định phân phối (giảm skewness), tốt cho mô hình tuyến tính.
<code>red_flag</code>	Cờ tổng hợp nhiều dấu hiệu bất thường (Feature Engineering dạng giám sát yếu).
<code>hour</code>	Thông tin chu kỳ thời gian theo giờ.
<code>day</code>	Thông tin chu kỳ thời gian theo ngày.
<code>isFlaggedFraud</code>	Cờ đánh dấu gian lận từ hệ thống quy tắc cũ (nếu có dữ liệu).
<code>isMerchant</code>	Thông tin xác định tài khoản là Merchant (nếu có ý nghĩa phân loại).
<code>isCustomerDest</code>	Thông tin xác định tài khoản đích là Khách hàng cá nhân.

2.5 Drop cột không cần thiết

Lý do drop sớm:

2.6 Danh sách Biến nên BỎ (Drop Features)

Các biến dưới đây được đề xuất loại bỏ vì nguy cơ gây nhiễu, rò rỉ thông tin (data leakage) hoặc dư thừa do đã được Feature Engineering sang các biến khác.

Tên Biến (Variable)	Lý do loại bỏ
<code>nameOrig</code>	Biến định danh (ID). Số lượng giá trị duy nhất quá lớn, dễ gây Overfitting .
<code>nameDest</code>	Biến định danh (ID). Tương tự như trên, dễ gây Overfitting .
<code>oldbalanceOrg</code>	Dư thừa. Đã được sử dụng để tạo: <code>balance_diff</code> , <code>amount_ratio_org</code> , <code>is_empty_orig</code> .
<code>newbalanceOrig</code>	Dư thừa. Đã được sử dụng để tạo biến <code>balance_diff</code> .
<code>oldbalanceDest</code>	Dư thừa. Đã được sử dụng để tạo: <code>errorBalanceDest</code> , <code>is_empty_dest</code> .
<code>newbalanceDest</code>	Dư thừa. Đã được sử dụng để tạo biến <code>errorBalanceDest</code> .
<code>step</code>	Dư thừa. Đã được chuyển đổi thành biến thời gian có ý nghĩa hơn là <code>hour</code> và <code>day</code> .

2.6.1 Lưu ý quan trọng khi Huấn luyện (Modeling Notes)

Sau khi xử lý biến (Feature Selection), cần lưu ý chiến lược cho từng loại mô hình:

- Với Logistic Regression:** * Cần Chuẩn hóa (Standard-Scaler/MinMaxScaler) các biến liên tục như: `log_amount`, `amount_ratio_org`, `abs_balance_diff`. * Mô hình tuyến tính rất nhạy cảm với độ lớn của dữ liệu (scale).
- Với Random Forest / XGBoost / LightGBM:** * Có thể sử dụng trực tiếp các biến gốc, biến nhị phân và tỷ lệ mà không cần chuẩn hóa khắt khe. * Mô hình cây xử lý tốt các quan hệ phi tuyến.
- Kiểm tra Đa cộng tuyến (Multicollinearity):** * Nếu giữ cả `balance_diff` và `abs_balance_diff`, mô hình tuyến tính có thể bị ảnh hưởng. * **Hành động:** Kiểm tra chỉ số **VIF (Variance Inflation Factor)**. Nếu $VIF > 5$ hoặc 10, cân nhắc loại bỏ một trong hai hoặc sử dụng **PCA**.
- Đánh giá độ quan trọng (Feature Importance):** * Sau khi train vòng đầu tiên (baseline), hãy plot biểu đồ **Feature Importance**. * Mạnh dạn loại bỏ các biến có đóng góp ~ 0 để làm nhẹ mô hình.

```
[42]: # Các cột không dùng cho mô hình
drop_cols = [
    'Unnamed: 0',          # Index trùng, không thông tin
    'nameOrig',            # Định danh → leakage
    'nameDest',            # Định danh → leakage
    'oldbalanceOrg',       # Đã được biến đổi thành balance_diff,
    ↪ is_empty_orig, amount_ratio_org
    'newbalanceOrig',      # Đã được biến đổi thành balance_diff
    'oldbalanceDest',      # Đã được biến đổi thành errorBalanceDest,
    ↪ is_empty_dest
    'newbalanceDest',      # Đã được biến đổi thành errorBalanceDest
    'step',               # Đã được biến đổi thành hour, day
    'isFlaggedFraud',      # Nghi ngờ leakage mạnh (có thể là nhãn từ hệ thống,
    ↪ cũ)
    'balance_diff',
    'errorBalanceDest',
    'abs_balance_diff',
    'abs_errorBalanceDest',
    'red_flag'
]

df = df.drop(columns=[c for c in drop_cols if c in df.columns])

df
```

```
[42]:
```

	type	amount	isFraud	isMerchant	isCustomerDest	hour	day	\
0	CASH_IN	290670.88	0	0	1	17	16	
1	CASH_OUT	74969.02	0	0	1	10	14	
2	CASH_OUT	152359.15	0	0	1	10	1	

3	TRANSFER	553238.82	0	0	1	14	12
4	CASH_IN	432624.19	0	0	1	10	5
...	...	...	...	...	...	...	...
325929	CASH_OUT	34345.29	0	0	1	18	11
325930	PAYMENT	8234.77	0	1	0	19	15
325931	CASH_OUT	13499.45	0	0	1	19	9
325932	CASH_OUT	236678.98	0	0	1	9	15
325933	CASH_OUT	108989.28	0	0	1	11	16

	is_empty_orig	is_empty_dest	amount_ratio_org	log_amount
0	0	0	0.009495	12.579950
1	1	0	74969.020000	11.224844
2	1	0	152359.150000	11.934002
3	1	0	553238.820000	13.223547
4	0	0	22.876854	12.977627
...	...	...	...	...
325929	0	1	1.685162	10.444249
325930	0	1	30.163993	9.016242
325931	1	0	13499.450000	9.510478
325932	0	1	23.623014	12.374464
325933	0	0	0.322888	11.599014

[325934 rows x 11 columns]

2.7 Tách X / y và train-test split

```
[43]: X = df.drop(columns="isFraud")
      y = df["isFraud"]

      X_train, X_test, y_train, y_test = train_test_split(
          X, y,
          test_size=0.2,
          random_state=42,
          stratify=y  # quan trọng với data lệch lớp
      )
```

2.8 Phân loại feature

```
[44]: # Tự động phát hiện kiểu biến
      num_features = X.select_dtypes(include=["int64", "float64"]).columns
      cat_features = X.select_dtypes(include=["object", "category"]).columns
```

2.9 Preprocessing Pipeline

Numerical pipeline

StandardScaler (Logistic Regression cần scale)

```
[45]: num_transformer = Pipeline(steps=[
    ("scaler", StandardScaler())
])
```

Categorical pipeline

OneHotEncoder (drop first để tránh dummy trap)

```
[46]: cat_transformer = Pipeline(steps=[
    ("onehot", OneHotEncoder(
        handle_unknown="ignore",
        drop="first"
    ))
])
```

2.10 ColumnTransformer

```
[47]: preprocessor = ColumnTransformer(
    transformers=[
        ("num", num_transformer, num_features),
        ("cat", cat_transformer, cat_features),
    ]
)
```

3 Pipeline với Logistic Regression

```
[48]: model = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("clf", LogisticRegression(
        max_iter=1000,
        class_weight="balanced", # rất quan trọng với fraud
        solver="lbfgs"
    ))
])
```

3.1 Huấn luyện model

```
[49]: model.fit(X_train, y_train)
```

```
[49]: Pipeline(steps=[('preprocess',
    ColumnTransformer(transformers=[('num',
    Pipeline(steps=[('scaler',
    StandardScaler())])),
    Index(['amount', 'isMerchant',
    'isCustomerDest', 'hour', 'day',
    'is_empty_orig', 'is_empty_dest', 'amount_ratio_org', 'log_amount'],
    dtype='object'))],
```

```

OneHotEncoder(drop='first',
               handle_unknown='ignore'))]),
               Index(['type'],
dtype='object'))]]),
               ('clf',
               LogisticRegression(class_weight='balanced', max_iter=1000))])
('cat',
 Pipeline(steps=[('onehot',

```

4 Đánh giá mô hình

```

[50]: y_pred = model.predict(X_test)
      y_prob = model.predict_proba(X_test)[:, 1]

      print("Classification Report:")
      print(classification_report(y_test, y_pred))

      print("Confusion Matrix:")
      print(confusion_matrix(y_test, y_pred))

      print("ROC-AUC:", roc_auc_score(y_test, y_prob))

```

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.86	0.92	63544
1	0.15	0.94	0.26	1643
accuracy			0.86	65187
macro avg	0.57	0.90	0.59	65187
weighted avg	0.98	0.86	0.91	65187

Confusion Matrix:

```
[[54684  8860]
```

```
[  105 1538]]
```

ROC-AUC: 0.9671820845557465

5 Diễn giải mô hình

Lấy hệ số Logistic Regression

```

[51]: feature_names = model.named_steps["preprocess"].get_feature_names_out()
      coefficients = model.named_steps["clf"].coef_[0]

      coef_df = pd.DataFrame({
          "feature": feature_names,

```

```

    "coef": coefficients
}).sort_values(by="coef", ascending=False)

coef_df.head(10)

```

```

[51]:
      feature      coef
12  cat__type_TRANSFER  8.522105
9   cat__type_CASH_OUT  7.873048
2   num__isCustomerDest  1.342748
0   num__amount         1.055270
6   num__is_empty_dest  0.665748
4   num__day           0.463339
8   num__log_amount    -0.382909
3   num__hour          -0.468954
10  cat__type_DEBIT    -0.508177
7   num__amount_ratio_org -0.812982

```

6 Tuning Hyperparameter

Optuna objective: tune Logistic Regression bằng CV score Chọn scoring

Fraud/imbalanced: khuyến nghị roc_auc hoặc average_precision (PR-AUC).

Ưu tiên bắt fraud: tối ưu recall / f1.

Dùng ROC-AUC (ổn định và threshold-independent)

```

[52]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

def objective(trial):
    # Penalty/solver hợp lệ theo sklearn:
    # - liblinear: l1, l2
    # - saga: l1, l2, elasticnet
    solver = trial.suggest_categorical("solver", ["liblinear", "saga"])
    penalty = trial.suggest_categorical("penalty", ["l1", "l2"] if solver in
    ↪ ["liblinear", "saga"] else ["l2"])

    params = {
        "C": trial.suggest_float("C", 1e-3, 1e2, log=True),
        "solver": solver,
        "penalty": penalty,
        "max_iter": 3000,
        "class_weight": trial.suggest_categorical("class_weight", [None,
    ↪ "balanced"]),
        "random_state": 42
    }

    # saga supports elasticnet; optional: add l1_ratio

```

```

    if solver == "saga" and trial.suggest_categorical("use_elasticnet", [False,
↪True]):
        params["penalty"] = "elasticnet"
        params["l1_ratio"] = trial.suggest_float("l1_ratio", 0.05, 0.95)

    clf = LogisticRegression(**params)

    pipe = Pipeline(steps=[
        ("preprocess", preprocessor),
        ("clf", clf)
    ])

    scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring="roc_auc",
↪n_jobs=-1)
    return scores.mean()

```

Run Optuna

```

[53]: study = optuna.create_study(direction="maximize")
      study.optimize(objective, n_trials=50)

      print("Best ROC-AUC (CV):", study.best_value)
      print("Best params:", study.best_params)

```

```

[I 2025-12-27 15:35:38,529] A new study created in memory with name: no-
name-7ec7bd43-5af4-44e6-9282-37f534141d8c
[I 2025-12-27 15:35:53,204] Trial 0 finished with value: 0.9679912090066347 and
parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 0.0025287875845509252,
'class_weight': 'balanced'}. Best is trial 0 with value: 0.9679912090066347.
[I 2025-12-27 15:37:02,968] Trial 1 finished with value: 0.9683328259368583 and
parameters: {'solver': 'saga', 'penalty': 'l1', 'C': 0.003780640594287094,
'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 1 with
value: 0.9683328259368583.
[I 2025-12-27 15:37:12,266] Trial 2 finished with value: 0.9302496778135637 and
parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 0.0011302258571276183,
'class_weight': None}. Best is trial 1 with value: 0.9683328259368583.
[I 2025-12-27 15:37:20,273] Trial 3 finished with value: 0.968763820511408 and
parameters: {'solver': 'liblinear', 'penalty': 'l2', 'C': 0.18690159020725808,
'class_weight': 'balanced'}. Best is trial 3 with value: 0.968763820511408.
[I 2025-12-27 15:37:22,736] Trial 4 finished with value: 0.9415703136404187 and
parameters: {'solver': 'liblinear', 'penalty': 'l2', 'C': 0.002859244518545158,
'class_weight': None}. Best is trial 3 with value: 0.968763820511408.
[I 2025-12-27 15:42:30,891] Trial 5 finished with value: 0.9658020789303743 and
parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 13.868004364676299,
'class_weight': None}. Best is trial 3 with value: 0.968763820511408.
[I 2025-12-27 15:49:06,728] Trial 6 finished with value: 0.9656118679569289 and
parameters: {'solver': 'saga', 'penalty': 'l1', 'C': 17.05248487526939,
'class_weight': None, 'use_elasticnet': True, 'l1_ratio': 0.6534868887536318}.

```

Best is trial 3 with value: 0.968763820511408.

[I 2025-12-27 15:50:33,620] Trial 7 finished with value: 0.9656025559903141 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 0.1609279843370405, 'class_weight': None}. Best is trial 3 with value: 0.968763820511408.

[I 2025-12-27 15:50:42,712] Trial 8 finished with value: 0.9672916305907459 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 0.0013261482773727267, 'class_weight': 'balanced'}. Best is trial 3 with value: 0.968763820511408.

[I 2025-12-27 15:56:26,932] Trial 9 finished with value: 0.9656095445192674 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 22.39575289849751, 'class_weight': None, 'use_elasticnet': False}. Best is trial 3 with value: 0.968763820511408.

[I 2025-12-27 15:58:28,772] Trial 10 finished with value: 0.9687899322498599 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 0.3042914736868917, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.05230464170413107}. Best is trial 10 with value: 0.9687899322498599.

[I 2025-12-27 16:00:31,650] Trial 11 finished with value: 0.9687854141254124 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 0.2644156578189184, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.05695013970875371}. Best is trial 10 with value: 0.9687899322498599.

[I 2025-12-27 16:03:52,573] Trial 12 finished with value: 0.9688119090625719 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 1.151254434963679, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.08404991576616239}. Best is trial 12 with value: 0.9688119090625719.

[I 2025-12-27 16:07:22,757] Trial 13 finished with value: 0.96881563972845 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 3.043254353154307, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.05446835143018956}. Best is trial 13 with value: 0.96881563972845.

[I 2025-12-27 16:11:49,762] Trial 14 finished with value: 0.9688151936069639 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 2.0418921497113818, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.281948276479826}. Best is trial 13 with value: 0.96881563972845.

[I 2025-12-27 16:16:36,440] Trial 15 finished with value: 0.9688157624873981 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 2.4095697652207577, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.3600244582347487}. Best is trial 15 with value: 0.9688157624873981.

[I 2025-12-27 16:20:22,835] Trial 16 finished with value: 0.9688173343942609 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 2.9376635356874896, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.9473926004585568}. Best is trial 16 with value: 0.9688173343942609.

[I 2025-12-27 16:23:48,997] Trial 17 finished with value: 0.9688178344114663 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 99.44002244164683, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.8659953122442943}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 16:27:00,440] Trial 18 finished with value: 0.9688178044702639 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 85.83214091247473, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 16:30:14,626] Trial 19 finished with value: 0.9688178014760668 and

parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 93.75917001374901, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:10:06,074] Trial 20 finished with value: 0.9688178014760668 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 93.47358844108939, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:15:19,247] Trial 21 finished with value: 0.9688178014760668 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 93.67349078060616, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:20:45,969] Trial 22 finished with value: 0.9688176128468499 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 33.667482964981204, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:25:19,526] Trial 23 finished with value: 0.9686416932027679 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 0.04641824344555617, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:31:05,591] Trial 24 finished with value: 0.9688170918720337 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 7.624036720925272, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:36:46,776] Trial 25 finished with value: 0.968817747582173 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 54.157676583678395, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:44:01,705] Trial 26 finished with value: 0.9688171278012764 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 8.161958138818138, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:49:17,549] Trial 27 finished with value: 0.9688176936879851 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 41.91674952183526, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:54:15,436] Trial 28 finished with value: 0.9688168822842634 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 6.514238520681787, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 19:57:29,981] Trial 29 finished with value: 0.9685672053600592 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 0.030425161451123447, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 20:02:26,479] Trial 30 finished with value: 0.9688153972057515 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 0.7168271904154775, 'class_weight': 'balanced', 'use_elasticnet': True, 'l1_ratio': 0.9295060127674554}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 20:07:40,826] Trial 31 finished with value: 0.9688177775233756 and

parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 98.16263985910605, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 20:12:56,927] Trial 32 finished with value: 0.9688177685409028 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 58.34890278824433, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 20:17:37,113] Trial 33 finished with value: 0.9688176098527119 and parameters: {'solver': 'saga', 'penalty': 'l2', 'C': 30.283996815415165, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 20:22:49,707] Trial 34 finished with value: 0.9688178314173284 and parameters: {'solver': 'saga', 'penalty': 'l1', 'C': 97.45756290247796, 'class_weight': 'balanced', 'use_elasticnet': False}. Best is trial 17 with value: 0.9688178344114663.

[I 2025-12-27 20:39:12,319] Trial 35 finished with value: 0.9688214303381338 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 13.429916500221118, 'class_weight': 'balanced'}. Best is trial 35 with value: 0.9688214303381338.

[I 2025-12-27 20:47:08,533] Trial 36 finished with value: 0.9658018304209316 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 15.039960294877748, 'class_weight': None}. Best is trial 35 with value: 0.9688214303381338.

[I 2025-12-27 21:06:09,193] Trial 37 finished with value: 0.9688215411195878 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 42.37224313401878, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 21:15:13,665] Trial 38 finished with value: 0.9658027496299921 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 11.629988178537042, 'class_weight': None}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 21:26:47,696] Trial 39 finished with value: 0.9688214662667877 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 23.330556430780064, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 22:07:10,315] Trial 40 finished with value: 0.9657985548778004 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 5.817848578145473, 'class_weight': None}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 22:15:59,245] Trial 41 finished with value: 0.9688214662671412 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 23.1408699767891, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 22:25:01,073] Trial 42 finished with value: 0.9688214063851481 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 17.45927891303895, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 22:35:03,256] Trial 43 finished with value: 0.9688214782436338 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 22.226224941583308, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 22:44:26,826] Trial 44 finished with value: 0.9688214812379485 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 24.62968155874435, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 22:52:53,773] Trial 45 finished with value: 0.9688215261491926 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 29.34883471139423, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

[I 2025-12-27 23:00:06,333] Trial 46 finished with value: 0.9658094683641243 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 39.44880145826549, 'class_weight': None}. Best is trial 37 with value: 0.9688215411195878.
 [I 2025-12-27 23:22:18,263] Trial 47 finished with value: 0.9688212596736854 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 4.407685512798598, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.
 [I 2025-12-27 23:53:37,218] Trial 48 finished with value: 0.9688203644343375 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 0.9824329411354353, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.
 [I 2025-12-28 00:12:38,964] Trial 49 finished with value: 0.968821481237595 and parameters: {'solver': 'liblinear', 'penalty': 'l1', 'C': 21.50562031220629, 'class_weight': 'balanced'}. Best is trial 37 with value: 0.9688215411195878.

Best ROC-AUC (CV): 0.9688215411195878

Best params: {'solver': 'liblinear', 'penalty': 'l1', 'C': 42.37224313401878, 'class_weight': 'balanced'}

Fit best model trên train

```
[54]: best = study.best_params

# Rebuild best logistic params (must mirror objective logic)
best_solver = best["solver"]
best_penalty = best["penalty"]

log_params = {
    "C": best["C"],
    "solver": best_solver,
    "penalty": best_penalty,
    "max_iter": 3000,
    "random_state": 42,
    "class_weight": best.get("class_weight", None),
}

if best_penalty == "elasticnet":
    log_params["l1_ratio"] = best["l1_ratio"]

best_pipe = Pipeline(steps=[
    ("preprocess", preprocessor),
    ("clf", LogisticRegression(**log_params))
])

best_pipe.fit(X_train, y_train)
```

```
[54]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(transformers=[('num',
                                                         Pipeline(steps=[('scaler',
                                                         StandardScaler()))]),
```

```

Index(['amount', 'isMerchant',
'isCustomerDest', 'hour', 'day',
'is_empty_orig', 'is_empty_dest', 'amount_ratio_org', 'log_amount'],
dtype='object')),
('cat',
Pipeline(steps=[('onehot',
OneHotEncoder(drop='first',
handle_unknown='ignore'))])),
Index(['type'],
dtype='object')))),
('clf',
LogisticRegression(C=42.37224313401878,
class_weight='balanced', max_iter=3000,
penalty='l1', random_state=42,
solver='liblinear'))))

```

Score (probability) + metrics threshold-free

ROC-AUC/PR-AUC là threshold-independent → đánh giá “chất lượng score”.

```

[55]: y_prob = best_pipe.predict_proba(X_test)[: , 1]

print("ROC-AUC:", roc_auc_score(y_test, y_prob))
print("PR-AUC (Average Precision):", average_precision_score(y_test, y_prob))

```

ROC-AUC: 0.967184163044222

PR-AUC (Average Precision): 0.5360322959497514

Tune threshold

Quét threshold và chọn theo F1

```

[56]: from sklearn.metrics import precision_recall_curve

# 1. Precision-Recall curve
precision, recall, thresholds = precision_recall_curve(y_test, y_prob)

# 2. F1-score cho từng threshold (lưu ý: precision, recall có len =
↳ len(thresholds) + 1)

f1_scores = (2 * precision * recall) / (precision + recall + 1e-9)

# 3. Bỏ phần tử đầu (không có threshold)
f1_scores = f1_scores[1:]
precision = precision[1:]
recall = recall[1:]

# 4. Tạo DataFrame kết quả

```

```

df_thr = pd.DataFrame({
    "threshold": thresholds,
    "precision": precision,
    "recall": recall,
    "f1": f1_scores
})

# 5. Threshold tối ưu theo F1

best_row = df_thr.loc[df_thr["f1"].idxmax()]
best_threshold = float(best_row["threshold"])

print("Best threshold (PR-curve, max F1):", best_threshold)
print(best_row)

```

```

Best threshold (PR-curve, max F1): 0.9024194148491818
threshold    0.902419
precision    0.555295
recall       0.577602
f1           0.566229
Name: 63469, dtype: float64

```

“bắt fraud” (Recall constraint)

```

[57]: target_recall = 0.85
      cand = df_thr[df_thr["recall"] >= target_recall].sort_values("precision",
      ↪ascending=False).head(1)

      if len(cand) > 0:
          thr_recall = float(cand["threshold"].iloc[0])
          print("Threshold with recall >=", target_recall, ":", thr_recall)
          print(cand)

```

```

Threshold with recall >= 0.85 : 0.6823225051567459
      threshold precision  recall    f1
59110    0.682323   0.230224  0.850274  0.36234

```

Final evaluation tại threshold đã chọn

```

[58]: y_pred_final = (y_prob >= best_threshold).astype(int)

      print("Confusion matrix:\n", confusion_matrix(y_test, y_pred_final))
      print(classification_report(y_test, y_pred_final, digits=4))

```

```

Confusion matrix:
[[62783  761]
 [ 694  949]]

```

	precision	recall	f1-score	support
0	0.9891	0.9880	0.9885	63544

1	0.5550	0.5776	0.5661	1643
accuracy			0.9777	65187
macro avg	0.7720	0.7828	0.7773	65187
weighted avg	0.9781	0.9777	0.9779	65187

7 Resampling

```
[59]: ! pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (0.14.1)
Requirement already satisfied: numpy<3,>=1.25.2 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (2.2.6)
Requirement already satisfied: scipy<2,>=1.11.4 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (1.15.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (1.7.0)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (0.1.5)
Requirement already satisfied: joblib<2,>=1.2.0 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (1.5.1)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in
c:\users\hi\appdata\local\programs\python\python312\lib\site-packages (from
imbalanced-learn) (3.6.0)
```

[notice] A new release of pip is available: 23.2.1 -> 25.3

[notice] To update, run: python.exe -m pip install --upgrade pip

```
[60]: from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.combine import SMOTEENN, SMOTETomek
```

7.1 THỬ CÁC CHIẾN LƯỢC RESAMPLING KHÁC NHAU

```
[61]: def create_pipeline_with_resampling(strategy='smote', sampling_params=None):
    """
    Tạo pipeline với các chiến lược resampling khác nhau
    """
```

```

if sampling_params is None:
    sampling_params = {}

# Pipeline cơ bản
base_pipeline = ImbPipeline(steps=[
    ("preprocess", preprocessor),
    ("resample", None), # sẽ được thay thế
    ("clf", LogisticRegression(
        max_iter=3000,
        random_state=42
    ))
])

# Chọn chiến lược resampling
if strategy == 'smote':
    base_pipeline.set_params(resample=SMOTE(**sampling_params))
elif strategy == 'smote_enn':
    base_pipeline.set_params(resample=SMOTEENN(**sampling_params))
elif strategy == 'smote_tomek':
    base_pipeline.set_params(resample=SMOTETomek(**sampling_params))
elif strategy == 'undersample':
    base_pipeline.set_params(resample=RandomUnderSampler(**sampling_params))
elif strategy == 'none':
    base_pipeline.set_params(resample=None)
    # Nếu không resampling, dùng class_weight
    base_pipeline.set_params(clf__class_weight='balanced')

return base_pipeline

```

7.2 THỬ NHANH CÁC CHIẾN LƯỢC RESAMPLING (CHƯA TUNING)

```

[62]: strategies = [
    ('none', {}), # Chỉ dùng class_weight
    ('smote', {'random_state': 42}),
    ('smote_enn', {'random_state': 42}),
    ('smote_tomek', {'random_state': 42}),
    ('undersample', {'random_state': 42})
]

print("THỬ NHANH CÁC CHIẾN LƯỢC RESAMPLING (CHƯA TUNING)")

for strategy_name, params in strategies:
    print(f"\nĐang thử: {strategy_name.upper()}")
    pipe = create_pipeline_with_resampling(strategy_name, params)

# Huấn luyện

```

```

pipe.fit(X_train, y_train)

# Dự đoán
y_pred = pipe.predict(X_test)
y_prob = pipe.predict_proba(X_test)[:, 1]

# Tính metrics
roc_auc = roc_auc_score(y_test, y_prob)
pr_auc = average_precision_score(y_test, y_prob)
recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f" ROC-AUC: {roc_auc:.4f}")
print(f" PR-AUC: {pr_auc:.4f}")
print(f" Recall: {recall:.4f}")
print(f" Precision: {precision:.4f}")
print(f" F1-Score: {f1:.4f}")

```

THỬ NHANH CÁC CHIẾN LƯỢC RESAMPLING (CHƯA TUNING)

Đang thử: NONE

ROC-AUC: 0.9672
 PR-AUC: 0.5360
 Recall: 0.9361
 Precision: 0.1479
 F1-Score: 0.2555

Đang thử: SMOTE

ROC-AUC: 0.9671
 PR-AUC: 0.5366
 Recall: 0.9373
 Precision: 0.1479
 F1-Score: 0.2556

Đang thử: SMOTE_ENN

ROC-AUC: 0.9671
 PR-AUC: 0.5356
 Recall: 0.9367
 Precision: 0.1503
 F1-Score: 0.2590

Đang thử: SMOTE_TOMEK

ROC-AUC: 0.9671
 PR-AUC: 0.5368
 Recall: 0.9373
 Precision: 0.1481
 F1-Score: 0.2557

Đang thử: UNDERSAMPLE

ROC-AUC: 0.9658

PR-AUC: 0.5441

Recall: 0.9385

Precision: 0.1457

F1-Score: 0.2523

7.3 TUNING HYPERPARAMETERS VỚI RESAMPLING TỐT NHẤT

Dựa trên kết quả thử nhanh, chọn strategy tốt nhất

(Giả sử chọn 'smote' vì thường tốt nhất cho fraud detection)

```
[63]: best_strategy = 'SMOTE_ENN'

print(f"TUNING HYPERPARAMETERS VỚI {best_strategy.upper()}")

# Hàm objective mới với resampling
def objective_with_resampling(trial):
    # Hyperparameters cho Logistic Regression
    C = trial.suggest_float("C", 1e-4, 100, log=True)
    solver = trial.suggest_categorical("solver", ["lbfgs", "liblinear", "saga"])
    penalty = 'l2'

    # Nếu dùng saga, có thể dùng elasticnet
    if solver == 'saga':
        penalty = trial.suggest_categorical("penalty", ["l1", "l2",
↪ "elasticnet"])
        if penalty == 'elasticnet':
            l1_ratio = trial.suggest_float("l1_ratio", 0.1, 0.9)

    # Hyperparameters cho SMOTE (nếu dùng)
    if best_strategy == 'smote':
        k_neighbors = trial.suggest_int("k_neighbors", 3, 10)
        sampling_strategy = trial.suggest_float("sampling_strategy", 0.3, 0.9)

    # Tạo pipeline
    pipe = ImbPipeline(steps=[
        ("preprocess", preprocessor),
        ("resample", SMOTE(
            random_state=42,
            k_neighbors=k_neighbors if best_strategy == 'smote' else 5,
            sampling_strategy=sampling_strategy if best_strategy == 'smote'
↪ else 0.5
        )),
        ("clf", LogisticRegression(
            C=C,
```

```

        solver=solver,
        penalty=penalty,
        max_iter=3000,
        random_state=42,
        class_weight=None # Không dùng vì đã có resampling
    ))
])

# Nếu có elasticnet
if solver == 'saga' and penalty == 'elasticnet':
    pipe.set_params(clf__l1_ratio=l1_ratio)

# Cross-validation
scores = cross_val_score(
    pipe, X_train, y_train,
    cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42),
    scoring='roc_auc',
    n_jobs=-1
)

return scores.mean()

```

TUNING HYPERPARAMETERS VỚI SMOTE_ENN

7.4 Chạy Optuna với resampling

```

[64]: # Chạy Optuna với resampling
study_resample = optuna.create_study(direction="maximize")
study_resample.optimize(objective_with_resampling, n_trials=20,
    ↳ show_progress_bar=True)

print(f"\nBest ROC-AUC (với resampling): {study_resample.best_value:.6f}")
print(f"Best parameters (với resampling): {study_resample.best_params}")

```

[I 2025-12-28 01:19:31,438] A new study created in memory with name: no-name-6ee26c7e-616a-42f3-8287-d1e4f96d0dfa

```
0%|          | 0/20 [00:00<?, ?it/s]
```

[I 2025-12-28 01:19:58,895] Trial 0 finished with value: 0.9663115908654847 and parameters: {'C': 0.0021712190903600357, 'solver': 'liblinear'}. Best is trial 0 with value: 0.9663115908654847.

[I 2025-12-28 01:20:05,826] Trial 1 finished with value: 0.9684678541435899 and parameters: {'C': 24.40810360081601, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:26:22,869] Trial 2 finished with value: 0.9684515548058751 and parameters: {'C': 0.4737358552413082, 'solver': 'saga', 'penalty': 'l1'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:26:25,260] Trial 3 finished with value: 0.9552147493810631 and

parameters: {'C': 0.00016605516510446755, 'solver': 'lbfgs'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:26:36,400] Trial 4 finished with value: 0.9684676206038637 and parameters: {'C': 11.994740680350297, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:26:45,763] Trial 5 finished with value: 0.9682729870182172 and parameters: {'C': 0.05234321227211528, 'solver': 'lbfgs'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:26:56,432] Trial 6 finished with value: 0.9682281020696363 and parameters: {'C': 0.058775793059246954, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:27:07,080] Trial 7 finished with value: 0.9682282134513339 and parameters: {'C': 0.05881113250892652, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:27:15,851] Trial 8 finished with value: 0.9499690484430352 and parameters: {'C': 0.00017239721149058413, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:27:24,819] Trial 9 finished with value: 0.9539673261293649 and parameters: {'C': 0.00026385912838854575, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:33:52,321] Trial 10 finished with value: 0.9684551603222343 and parameters: {'C': 96.73991835378443, 'solver': 'saga', 'penalty': 'elasticnet', 'l1_ratio': 0.7325951919855392}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:33:55,599] Trial 11 finished with value: 0.9684677948598286 and parameters: {'C': 62.342323768769425, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:34:06,786] Trial 12 finished with value: 0.9684673529316435 and parameters: {'C': 8.188025221150005, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:34:17,870] Trial 13 finished with value: 0.9684678128245713 and parameters: {'C': 96.70319287749061, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:39:05,483] Trial 14 finished with value: 0.9684427934285749 and parameters: {'C': 1.3564204163870361, 'solver': 'saga', 'penalty': 'l2'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:39:09,039] Trial 15 finished with value: 0.9684417317811177 and parameters: {'C': 6.309820023016703, 'solver': 'lbfgs'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:39:15,335] Trial 16 finished with value: 0.9684678361787412 and parameters: {'C': 26.416356979776758, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:39:20,696] Trial 17 finished with value: 0.9684411927883073 and parameters: {'C': 0.719127189066119, 'solver': 'liblinear'}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:41:13,011] Trial 18 finished with value: 0.9678502500064442 and parameters: {'C': 0.007081628246267949, 'solver': 'saga', 'penalty': 'elasticnet', 'l1_ratio': 0.10114942786557518}. Best is trial 1 with value: 0.9684678541435899.

[I 2025-12-28 01:41:22,279] Trial 19 finished with value: 0.9684395095611452 and parameters: {'C': 19.83273943495561, 'solver': 'lbfgs'}. Best is trial 1 with value: 0.9684678541435899.

Best ROC-AUC (với resampling): 0.968468

Best parameters (với resampling): {'C': 24.40810360081601, 'solver': 'liblinear'}

7.5 HUẤN LUYỆN MÔ HÌNH CUỐI CÙNG VỚI PARAMS TỐT NHẤT

```
[65]: # Lấy best parameters
best_params = study_resample.best_params

# Tạo pipeline final với best parameters
final_pipe = ImbPipeline(steps=[
    ("preprocess", preprocessor),
    ("resample", SMOTE(
        random_state=42,
        k_neighbors=best_params.get('k_neighbors', 5),
        sampling_strategy=best_params.get('sampling_strategy', 0.5)
    )),
    ("clf", LogisticRegression(
        C=best_params['C'],
        solver=best_params['solver'],
        penalty=best_params.get('penalty', 'l2'),
        max_iter=3000,
        random_state=42,
        class_weight=None
    ))
])

# Thêm l1_ratio nếu có
if best_params.get('penalty') == 'elasticnet':
    final_pipe.set_params(clf__l1_ratio=best_params['l1_ratio'])

# Huấn luyện trên toàn bộ tập train
final_pipe.fit(X_train, y_train)
```

```
[65]: Pipeline(steps=[('preprocess',
                        ColumnTransformer(transformers=[('num',
                                                         Pipeline(steps=[('scaler',
                                                                              StandardScaler())])),
                                                         Index(['amount', 'isMerchant',
                                                                              'isCustomerDest', 'hour', 'day',
                                                                              'is_empty_orig', 'is_empty_dest', 'amount_ratio_org', 'log_amount'],
                                                                              dtype='object')),
                                                         ('cat',
```

```

Pipeline(steps=[('onehot',
OneHotEncoder(drop='first',
handle_unknown='ignore'))]),
Index(['type'],
dtype='object')))),
('resample', SMOTE(random_state=42, sampling_strategy=0.5)),
('clf',
LogisticRegression(C=24.40810360081601, max_iter=3000,
random_state=42, solver='liblinear'))])

```

7.6 TUNING THRESHOLD CHO MÔ HÌNH RESAMPLING

```

[66]: # Lấy probabilities
y_prob_resample = final_pipe.predict_proba(X_test)[: , 1]

# Tìm threshold tối ưu theo F1-score
precisions, recalls, thresholds = precision_recall_curve(y_test,
↳ y_prob_resample)

# Tính F1-score cho từng threshold
f1_scores = 2 * (precisions[:-1] * recalls[:-1]) / (precisions[:-1] + recalls[:
↳ -1] + 1e-10)

# Tìm threshold tốt nhất
best_idx = np.argmax(f1_scores)
best_threshold_resample = thresholds[best_idx]
best_f1 = f1_scores[best_idx]
best_precision = precisions[best_idx]
best_recall = recalls[best_idx]

print(f"Best threshold (max F1): {best_threshold_resample:.4f}")
print(f" F1-score: {best_f1:.4f}")
print(f" Precision: {best_precision:.4f}")
print(f" Recall: {best_recall:.4f}")

```

```

Best threshold (max F1): 0.8396
F1-score: 0.5678
Precision: 0.5707
Recall: 0.5648

```

7.7 ĐÁNH GIÁ MÔ HÌNH RESAMPLING VỚI THRESHOLD TỐI ƯU

```

[67]: # Dự đoán với threshold tối ưu
y_pred_resample = (y_prob_resample >= best_threshold_resample).astype(int)

# Tính metrics
roc_auc_resample = roc_auc_score(y_test, y_prob_resample)

```

```

pr_auc_resample = average_precision_score(y_test, y_prob_resample)

print("Classification Report:")
print(classification_report(y_test, y_pred_resample, digits=4))

print(f"\nROC-AUC: {roc_auc_resample:.6f}")
print(f"PR-AUC: {pr_auc_resample:.6f}")

print("\nConfusion Matrix:")
cm_resample = confusion_matrix(y_test, y_pred_resample)
print(cm_resample)

```

Classification Report:

	precision	recall	f1-score	support
0	0.9888	0.9890	0.9889	63544
1	0.5707	0.5648	0.5678	1643
accuracy			0.9783	65187
macro avg	0.7797	0.7769	0.7783	65187
weighted avg	0.9782	0.9783	0.9783	65187

ROC-AUC: 0.966733

PR-AUC: 0.542437

Confusion Matrix:

```

[[62846  698]
 [  715  928]]

```

7.8 SO SÁNH KẾT QUẢ TRƯỚC VÀ SAU RESAMPLING

```

[68]: # Lấy kết quả từ model gốc (không resampling)
y_pred_original = model.predict(X_test)
y_prob_original = model.predict_proba(X_test)[:, 1]

# Tính metrics cho model gốc
roc_auc_original = roc_auc_score(y_test, y_prob_original)
pr_auc_original = average_precision_score(y_test, y_prob_original)
recall_original = recall_score(y_test, y_pred_original)
precision_original = precision_score(y_test, y_pred_original)
f1_original = f1_score(y_test, y_pred_original)

# Tính metrics cho model với resampling
recall_resample = recall_score(y_test, y_pred_resample)
precision_resample = precision_score(y_test, y_pred_resample)
f1_resample = f1_score(y_test, y_pred_resample)

```

```

# Tạo bảng so sánh
comparison_df = pd.DataFrame({
    'Metric': ['ROC-AUC', 'PR-AUC', 'Recall', 'Precision', 'F1-Score'],
    'Original (no resampling)': [
        f'{roc_auc_original:.6f}',
        f'{pr_auc_original:.6f}',
        f'{recall_original:.4f}',
        f'{precision_original:.4f}',
        f'{f1_original:.4f}'
    ],
    f'With {best_strategy.upper()}': [
        f'{roc_auc_resample:.6f}',
        f'{pr_auc_resample:.6f}',
        f'{recall_resample:.4f}',
        f'{precision_resample:.4f}',
        f'{f1_resample:.4f}'
    ],
    'Improvement': [
        f'{{{roc_auc_resample - roc_auc_original} / roc_auc_original * 100}:.
↪2f}% ',
        f'{{{pr_auc_resample - pr_auc_original} / pr_auc_original * 100}:.2f}% ',
        f'{{{recall_resample - recall_original} / recall_original * 100}:.2f}% ',
        f'{{{precision_resample - precision_original} / precision_original *
↪100}:.2f}% ',
        f'{{{f1_resample - f1_original} / f1_original * 100}:.2f}% '
    ]
})

print(comparison_df.to_string(index=False))

```

Metric	Original (no resampling)	With SMOTE_ENN	Improvement
ROC-AUC	0.967182	0.966733	-0.05%
PR-AUC	0.536025	0.542437	1.20%
Recall	0.9361	0.5648	-39.66%
Precision	0.1479	0.5707	285.85%
F1-Score	0.2555	0.5678	122.25%

7.9 VẼ BIỂU ĐỒ SO SÁNH

```
[69]: fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```

# ROC Curve
from sklearn.metrics import roc_curve
fpr_orig, tpr_orig, _ = roc_curve(y_test, y_prob_original)
fpr_res, tpr_res, _ = roc_curve(y_test, y_prob_resample)

```

```

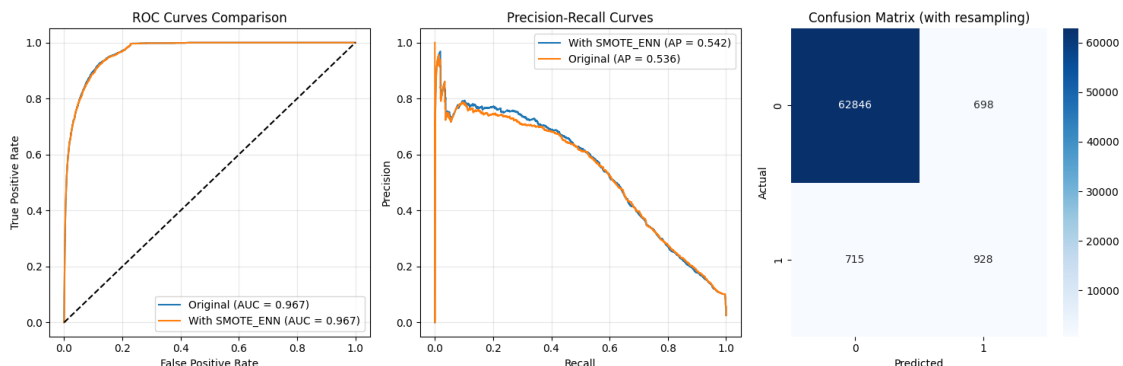
axes[0].plot(fpr_orig, tpr_orig, label=f'Original (AUC = {roc_auc_original:.3f})')
axes[0].plot(fpr_res, tpr_res, label=f'With {best_strategy.upper()} (AUC = {roc_auc_resample:.3f})')
axes[0].plot([0, 1], [0, 1], 'k--')
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('ROC Curves Comparison')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Precision-Recall Curve
axes[1].plot(recalls[:-1], precisions[:-1], label=f'With {best_strategy.upper()} (AP = {pr_auc_resample:.3f})')
precision_orig, recall_orig, _ = precision_recall_curve(y_test, y_prob_original)
axes[1].plot(recall_orig, precision_orig, label=f'Original (AP = {pr_auc_original:.3f})')
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curves')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Confusion Matrix Heatmap
sns.heatmap(cm_resample, annot=True, fmt='d', cmap='Blues', ax=axes[2])
axes[2].set_xlabel('Predicted')
axes[2].set_ylabel('Actual')
axes[2].set_title('Confusion Matrix (with resampling)')

plt.tight_layout()
plt.show()

```



7.10 DIỄN GIẢI MÔ HÌNH (FEATURE IMPORTANCE)

```
[70]: # Lấy tên feature và hệ số
feature_names = final_pipe.named_steps['preprocess'].get_feature_names_out()
coefficients = final_pipe.named_steps['clf'].coef_[0]

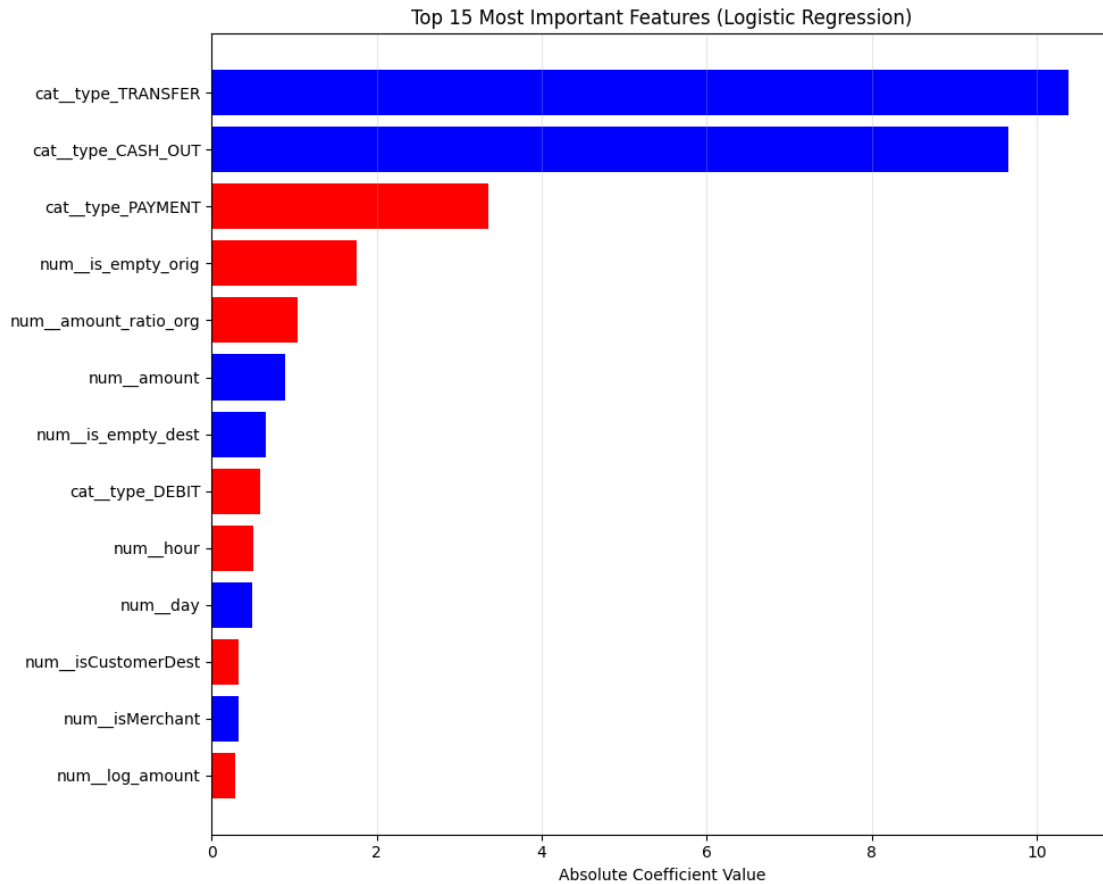
# Tạo DataFrame
feature_importance_df = pd.DataFrame({
    'feature': feature_names,
    'coefficient': coefficients,
    'abs_coefficient': np.abs(coefficients)
})

# Sắp xếp theo độ lớn của hệ số
feature_importance_df = feature_importance_df.sort_values('abs_coefficient',
    ↪ascending=False)

# Hiển thị top 15 features
print(feature_importance_df.head(15).to_string(index=False))

# Vẽ biểu đồ
plt.figure(figsize=(10, 8))
top_features = feature_importance_df.head(15)
colors = ['red' if coef < 0 else 'blue' for coef in top_features['coefficient']]
plt.barh(top_features['feature'], top_features['abs_coefficient'], color=colors)
plt.xlabel('Absolute Coefficient Value')
plt.title('Top 15 Most Important Features (Logistic Regression)')
plt.gca().invert_yaxis()
plt.grid(True, alpha=0.3, axis='x')
plt.tight_layout()
plt.show()
```

	feature	coefficient	abs_coefficient
	cat__type_TRANSFER	10.391307	10.391307
	cat__type_CASH_OUT	9.658919	9.658919
	cat__type_PAYMENT	-3.354144	3.354144
	num__is_empty_orig	-1.751952	1.751952
num__amount_ratio_org		-1.042509	1.042509
	num__amount	0.881736	0.881736
	num__is_empty_dest	0.660556	0.660556
	cat__type_DEBIT	-0.589084	0.589084
	num__hour	-0.509181	0.509181
	num__day	0.494363	0.494363
num__isCustomerDest		-0.325283	0.325283
	num__isMerchant	0.325283	0.325283
	num__log_amount	-0.281460	0.281460



7.11 LƯU MÔ HÌNH TỐT NHẤT

```
[71]: import joblib
import os

# Tạo thư mục models nếu chưa có
os.makedirs('models', exist_ok=True)

# Lưu cả preprocessor và model
joblib.dump(final_pipe, 'C:/Users/hi/Downloads/fraud_detection_project/models/
↳logistic_regression_resampled.pkl')
print(f"\nModel đã được lưu tại: C:/Users/hi/Downloads/fraud_detection_project/
↳models/logistic_regression_resampled.pkl")

# Lưu threshold tối ưu
threshold_data = {
    'best_threshold': best_threshold_resample,
    'best_f1': best_f1,
    'best_precision': best_precision,
```

```

    'best_recall': best_recall
}

joblib.dump(threshold_data, 'C:/Users/hi/Downloads/fraud_detection_project/
↳models/best_threshold.pkl')
print(f"Threshold tối ưu đã được lưu: {best_threshold_resample:.4f}")

```

Model đã được lưu tại: C:/Users/hi/Downloads/fraud_detection_project/models/logistic_regression_resampled.pkl

Threshold tối ưu đã được lưu: 0.8396

7.11.1 Kết luận & Phân tích Logistic Regression

1. Hiệu suất mô hình (với SMOTE+ENN): * **ROC-AUC:** 0.9667 * **F1-Score:** 0.5678 (Cải thiện **122%** so với model gốc không resampling). * **Nhận xét:** Mặc dù Recall cải thiện, nhưng Precision vẫn thấp (~0.57). Điều này cho thấy ranh giới giữa gian lận và giao dịch thường không hoàn toàn tuyến tính.

2. Phân tích trọng số (Feature Importance): > Các biến có hệ số (coefficient) lớn nhất: > * **type_TRANSFER** (+10.39) > * **type_CASH_OUT** (+9.65) > > **Insight:** Gian lận hầu như chỉ xảy ra ở hai loại giao dịch này. Bất kỳ giao dịch nào thuộc nhóm này đều có xác suất gian lận cao hơn hẳn về mặt tuyến tính.

```

[5]: # Recreate the SAME split
X = df.drop(columns="isFraud")
y = df["isFraud"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Save test set for evaluation
X_test.to_csv("C:/Users/hi/Downloads/fraud_detection_project/data/processed/
↳X_test.csv", index=False)
y_test.to_csv("C:/Users/hi/Downloads/fraud_detection_project/data/processed/
↳y_test.csv", index=False)

```